

Practice Problems: File systems

1. Provide one reason why a DMA-enabled device driver usually gives better performance over a non-DMA interrupt-driven device driver.

Ans: A DMA driver frees up CPU cycles that would have been spent copying data from the device to physical memory.

2. Consider a file D1/F1 that is hard linked from another parent directory D2. Then the directory entry of this file (including the filename and inode number) in directory D1 must be exactly identical to the directory entry in directory D2. [T/F]

Ans: F (the file name can be different)

3. Reading files via memory mapping them avoids an extra copy of file data from kernel space buffers to user space buffers. [T/F]

Ans: T

4. A soft link can create a link between files across different file systems, whereas a hard link can only create links between a directory and a file within the same file system. [T/F]

Ans: T (because hard link stores inode number, which is unique only within a file system)

5. Consider the process of opening a new file that does not exist (obviously, creating it during opening), via the “open” system call. Describe changes to all the in-memory and disk-based file system structures (e.g., file tables, inodes, and directories) that occur as part of this system call implementation. Write clearly, listing the structure that is changed, and the change made to it.

Ans: (a) New inode allocated on disk (with link count=1), and inode bitmap updated in the process. (b) Directory entry added to parent directory, to add mapping from file name to inode number. (c) In-memory inode allocated. (d) System-wide open file table points to in-memory inode. (e) Per-process file descriptor table points to open file table entry.

6. Now, suppose the process that has opened the file in the previous question proceeds to write 100 bytes into the file. Assume block size on disk is 512 bytes. Assume the OS uses a write-through disk buffer cache. List all the operations/changes to various datastructures that take place when the write operation successfully completes.

Ans: (a) open file table offset is changed (b) in-memory and on-disk inode adds pointer to new data block, and last modified time is updated (c) a copy of the data block comes into the disk buffer cache (d) New data block is allocated from data block bitmap (e) new data block is filled with user provided data

7. Repeat the above question for the implementation of the “link” system call, when linking to an existing file (not open from any process) in a directory from another new parent directory.

Ans: (a) The link count of the on-disk inode of the file is incremented. (b) A directory entry is added to the new directory to create a mapping from the file name to the inode number of the original file (if the new directory does not have space in its data blocks for the new file, a new data block is allocated for the new directory entry, and a pointer to this data block is added from the directory’s inode).

8. Repeat the above question for the implementation of the “dup” system call on a file descriptor.

Ans: To dup a file descriptor, another empty slot in the file descriptor table of the process is found, and this new entry is set to point to the same global open file table entry as the old file descriptor. That is, two FDs point to same system-wide file table entry.

9. Consider a file system with 512-byte blocks. Assume an inode of a file holds pointers to N direct data blocks, and a pointer to a single indirect block. Further, assume that the single indirect block can hold pointers to M other data blocks. What is the maximum file size that can be supported by such an inode design?

Ans: $(N+M)*512$ bytes

10. Consider a FAT file system where disk is divided into M byte blocks, and every FAT entry can store an N bit block number. What is the maximum size of a disk partition that can be managed by such a FAT design?

Ans: $2^N * M$ bytes

11. Consider a secondary storage system of size 2 TB, with 512-byte sized blocks. Assume that the filesystem uses a multilevel inode datastructure to track data blocks of a file. The inode has 64 bytes of space available to store pointers to data blocks, including a single indirect block, a double indirect block, and several direct blocks. What is the maximum file size that can be stored in such a file system?

Ans: Number of data blocks = $2^{41}/2^9 = 2^{32}$, so 32 bits or 4 bytes are required to store the number of a data block.

Number of data block pointers in the inode = $64/4 = 16$, of which 14 are direct blocks. The single indirect block stores pointers to $512/4 = 128$ data blocks. The double indirect block points to 128 single indirect blocks, which in turn point to 128 data blocks each.

So, the total number of data blocks in a file can be $14 + 128 + 128*128 = 16526$, and the maximum file size is $16526*512$ bytes.

12. Consider a filesystem managing a disk with block size 2^b bytes, and disk block addresses of 2^a bytes. The inode of a file contains n direct blocks, one single indirect block, one double indirect block, and one triple indirect block. What is the maximum size of a file (in bytes) that can be stored in this filesystem? Assume that the indirect blocks only store a sequence of disk addresses, and no other metadata.

Ans: Let x = number of disk addresses per block = 2^{b-a} . Then max file size is $2^b * (n + x + x^2 + x^3)$.

13. The `fork` system call creates new entries in the open file table for the newly created child process. [T/F]

Ans: F

14. When a process opens a file that is already being read by another process, the file descriptors in both processes will point to the same open file table entry. [T/F]

Ans: F

15. Memory mapping a file using the `mmap` system call adds one or more entries to the page table of the process. [T/F]

Ans: T

16. The read system call to fetch data from a file always blocks the invoking process. [T/F]

Ans: F (the data may be readily available in the disk buffer cache)

17. During filesystem operations, if the filesystem implementation ensures that changes to data blocks of a file are flushed to disk before changes to metadata blocks (like inodes and bitmaps), then the filesystem will never be in an inconsistent state after a crash, and a filesystem checker need not be run to detect and fix any inconsistencies. [T/F]

Ans: F (If there are multiple metadata operations, some may have happened and some may have been lost, causing an inconsistency. For example, a bitmap may indicate a data block is allocated but no inode points to it.)

18. Interrupt-based device drivers give superior performance to polling-based drivers because they eliminate the time spent by the CPU in copying data to and from the device hardware. [T/F]

Ans: F

19. When a process writes a block to the disk via a disk buffer cache using the write-back policy, the process invoking the write will block until the write is committed to disk. [T/F]

Ans: F

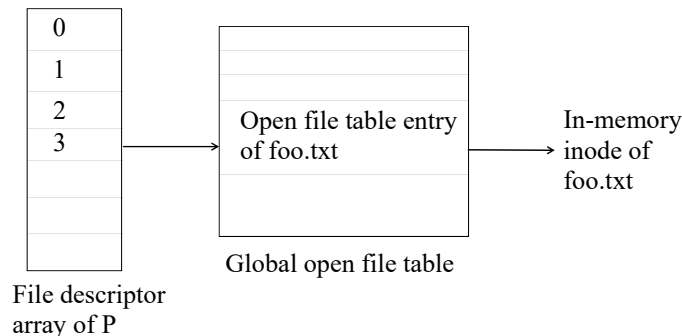
20. Consider a process P that has opened a file `foo.txt` using the `open` system call. The figure below shows the file descriptor array of P and the global open file table, and the pointers linking these data structures.

(a) After opening the file, P forks a child C. Draw a figure showing the file descriptor arrays of P and C, and the global open file table, immediately after the fork system call successfully completes. It is enough to show the entries pertaining to the file `foo.txt`, as in the figure above.

(b) Repeat part (a) for the following scenario: after P forks a child C, another process Q also opens the same file `foo.txt`.

Ans: In (a), the file descriptor arrays of P and C are pointing to the same file table entry. In (b), the file descriptor array of Q is pointing to a new open file table entry, which points to the same inode of the file `foo.txt`.

21. Consider a simple online banking application that stores all information about the accounts of users, like the money balance in their accounts, as fixed-size records on a big file on the hard disk. Let B_K denote the block number on the disk where the information about the account of any user



K is stored. Now, a user X wishes to transfer N rupees to the account of user Y . This transaction involves deducting N from the account of X and adding it to the account of Y (in this order) inside the banking application, which is accomplished by two write system calls to change the two disk blocks B_X and B_Y , with a seek system call in between. You may assume that there is enough money in the account of X to allow this transaction to happen. You may also assume that the application knows the offset values in the file where each account is located. The filesystem uses a write-through disk buffer cache.

- (a) Now suppose a power failure happens during the execution of this transaction. Describe the possible inconsistencies that the users of the application may perceive after the system restarts, due to a partial execution of the transaction.

Ans: X may see the amount deducted, but the amount does not appear in Y 's account, because the first write has reached the disk, but not the second one.

- (b) Continuing from the previous question, describe one idea by which we can avoid these inconsistencies, and ensure that, under power failures or crashes, either all or none of the steps of a transaction are executed.

Ans: We can use some kind of journaling or logging, to write both changes to a log first, before applying them to the actual disk blocks.

22. Consider a simple filesystem where an inode of a file stores 10 direct block numbers, and a single indirect block number. The indirect block is filled entirely with block numbers of subsequent blocks of the file. Each block number is 8 bytes in size. The block size in the system is 512 bytes. A directory entry in the directory data blocks (containing the mapping from file name to inode number) is 16 bytes in size. The directory entries are stored contiguously in the directory data blocks. Using this information, calculate the following maximum limits in the filesystem.

- (a) Maximum file size (in bytes) that can be stored in the filesystem

Ans: The indirect block has $512/8 = 64$ block numbers. So each file can have a maximum of 10 direct + 64 indirect block numbers = 74 blocks. Maximum file size is $74 * 512$ bytes = 37888 bytes.

(b) Maximum number of files that can be stored in a directory

Ans: A directory, like a file, can have a maximum of 74 data blocks. Each directory data block has $512/16 = 32$ directory entries, so we can store a maximum of $74 * 32 = 2368$ files in a directory.

23. Consider the following pseudocode of the `read` system call in a simple OS. Fill in the **five** blanks below. An explanation of the functions and variables used is provided after the code.

```
read(fd, dst, n):
    inode = fdi(fd)
    done = 0
    while(_____) { //fill
        buf = blkread(blkmap(inode, _____)) //fill
        memcpy( _____ , _____ , _____ ) //fill 3
        offset += BLKSIZE, done += BLKSIZE
    }
```

The arguments to the function are as follows. `fd` is the open file descriptor. `dst` is a `char *` pointer to the start of a userspace character buffer to read into. `n` is the total number of bytes to be read. Assume that the read system call succeeds, and `n` bytes to read are present in the file.

The other variables in the code are as follows. `inode` is a pointer to the in-memory inode of the file. `done` is a local variable to count the number of bytes that have been read so far. `buf` is a `char *` pointer to the data of a block in the disk buffer cache. `offset` is the current offset in bytes in the file stream. The constant `BLKSIZE` denotes the block size in bytes in the system. Assume that `n` and `offset` are always integer multiples of `BLKSIZE`, that is, the file is always read in multiples of `BLKSIZE`.

The functions invoked in the code are as follows. `fdi(fd)` returns the inode corresponding to a file descriptor `fd`. `blkread(K)` returns a `char *` pointer to the data of the disk block number `K` in the disk buffer cache (reading it from disk to cache if required). `blkmap(inode, K)` returns the block number of the `K`-th block of the file. For example, `blkmap(inode, 0)` returns the first block number of the file, `blkmap(inode, 1)` returns the second block number and so on. `memcpy(src, dst, num)` copies `num` bytes from `char *src` to `char *dst`.

Ans: `done < n`
`offset/BLKSIZE`
`memcpy(buf, dst+done, BLKSIZE)`

24. Consider the following pseudocode to implement the `link` system call for hard linking a file in a simple OS. Fill in the **five** blanks below. An explanation of the functions and variables used is provided after the code.

```
link(oldfilename, newfilename):
    inode = ilookup(_____) //fill
```

```
dip = ilookup_parent(_____) //fill
dirlink(dip , _____ , _____ ) //fill 2
inode.linkcnt = _____ //fill
```

The arguments to the function are the old and new filenames, and the function adds a hard link to the oldfilename from the new alias newfilename. Assume that the system call succeeds and no errors are encountered. The code shown above omits checking for error conditions as well.

The other variables in the code are as follows. `inode` is the in-memory inode structure, which has two elements: the inode number `inode.num` and the link count `inode.linkcnt`. Similarly, `dip` is the directory's in-memory inode structure.

The functions invoked in the code are as follows. `ilookup` returns the inode corresponding to a filename. `ilookup_parent` returns the inode of the parent directory corresponding to the filename. `dirlink(dir_inode, filename, inode_num)` add a directory entry in the directory corresponding to `dir_inode`, linking filename to `inode_num`.

Ans: oldfilename
newfilename
newfilename
inode.num
inode.linkcnt + 1

25. Consider the following (jumbled) set of events that may occur during a disk read by a process.

E1: Process in kernel mode busily polls for the completion of the disk read

E2: Disk data is copied from device memory to main memory

E3: The OS device driver issues a command to the device to begin the disk read

E4: Disk raises an interrupt

E5: Process that made the read system call is blocked and switched out of the CPU

For each of the settings of the device driver given below, write the events from the list above that occur in the scenario, in the chronological order in which they occur (earliest to latest).

- (a) Polling based device driver that polls without blocking the process

Ans: E3, E1, E2

- (b) Interrupt based driver without DMA

Ans: E3, E5, E4, E2

- (c) Interrupt based driver with DMA

Ans: E3, E5, E2, E4